Introduction to Big Data

Assignment 2

Vladislav Kalinichenko DS-02

Methodology

System Architecture

Three Docker containers handle different parts of the pipeline. cluster-master is where Hadoop NameNode, YARN ResourceManager, and all driver-side code run. cluster-slave-1 runs a DataNode and a NodeManager, so actual HDFS blocks live there and MapReduce tasks execute there. cassandra-server runs ScyllaDB that stores the finished inverted index. The app/ directory is mounted as a volume into /app on both cluster containers, so all scripts and Python files are accessible without any extra steps.

Data Preparation

The Kaggle Wikipedia 20230701 dataset is a large Parquet file with hundreds of thousands of articles. Following the task requirements, I loaded it with Spark and took a random sample of exactly 1,000 articles using a fixed seed for reproducibility. Each article gets saved as a .txt file in data/ with the format `<id>_<title>.txt`. These files then go to HDFS /data/ for use in indexing and for title lookups during search.

MapReduce Pipeline – Inverted Index Construction

I built the index with a single Hadoop Streaming job.

Mapper reads each .txt document from HDFS /data/. It gets the doc_id from the input filename via the `mapreduce_map_input_file` environment variable that Hadoop sets per mapper task. The mapper tokenizes the text with the regex `[a-zA-Z]+`, lowercases it, and counts term frequencies with `collections.Counter`. Since BM25 needs each document's total length for its length normalization term, the mapper also computes dl – the total token count – and includes it in the output. Output per unique term: `term<TAB>doc_id<TAB>tf<TAB>dl`.

Reducer receives mapper output already sorted by term, so it groups postings for each term in a single pass. It collects all doc_id, tf, and dl entries per term, counts df as the number of unique documents containing the term, and outputs: `term<TAB>df<TAB>doc_id1:tf1:dl1,doc_id2:tf2:dl2,...`

The job runs with a single reducer (`-D mapreduce.job.reduces=1`) so all postings for a given term end up in one output file /indexer/part-00000.

ScyllaDB Schema Design

ScyllaDB stores the index in three tables in the search_engine keyspace:

```
CREATE TABLE inverted_index (
    term    TEXT,
    doc_id  INT,
```

```sql
    tf      INT,
    PRIMARY KEY (term, doc_id)
);

CREATE TABLE doc_stats (
    doc_id  INT PRIMARY KEY,
    title   TEXT,
    dl      INT
);

CREATE TABLE global_stats (
    key     TEXT PRIMARY KEY,
    value   DOUBLE
);
```

inverted_index is partitioned by term so all postings for one query term land on the same node. doc_stats is partitioned by doc_id for fast title and document length lookups when displaying results. global_stats holds N – total documents – and dl_avg – average document length – the two corpus-level values BM25 needs.

I chose ScyllaDB over Apache Cassandra because it runs well on lower memory, which matters in a Docker environment. The scylla-driver Python package works as a direct replacement for cassandra-driver with no changes to query logic.

store_index.py reads /indexer/part-00000 from HDFS, parses each posting list, and inserts records into all three tables with prepared statements. It also computes N and dl_avg from the data and writes them to global_stats.

BM25 Search with PySpark

query.py implements BM25 ranking with the PySpark RDD API, using k = 1.0 and b = 0.75:

$$\mathrm{BM25}(q,d) = \sum_{t \in q} \log\left(\frac{N}{df(t)}\right) \cdot \frac{(k_1+1)\cdot tf(t,d)}{k_1 \cdot \left[(1-b) + b \cdot \frac{dl(d)}{dl_{avg}}\right] + tf(t,d)}$$

The driver tokenizes the query, connects to ScyllaDB, and fetches global_stats, doc_stats, and postings from inverted_index for all query terms. That data gets broadcast to all executors via sc.broadcast(). An RDD gets built from the postings list, each record maps to a doc_id with a partial BM25 score, reduceByKey sums scores per document, and the top 10 results print with titles and scores.

Demonstration

Running the System

```
docker compose up
```

Everything runs automatically through app.sh. Hadoop services start first, then a Python virtual environment gets created and packed with venv-pack, then data preparation, MapReduce indexing, index storage in ScyllaDB, and three sample queries.

Screenshots

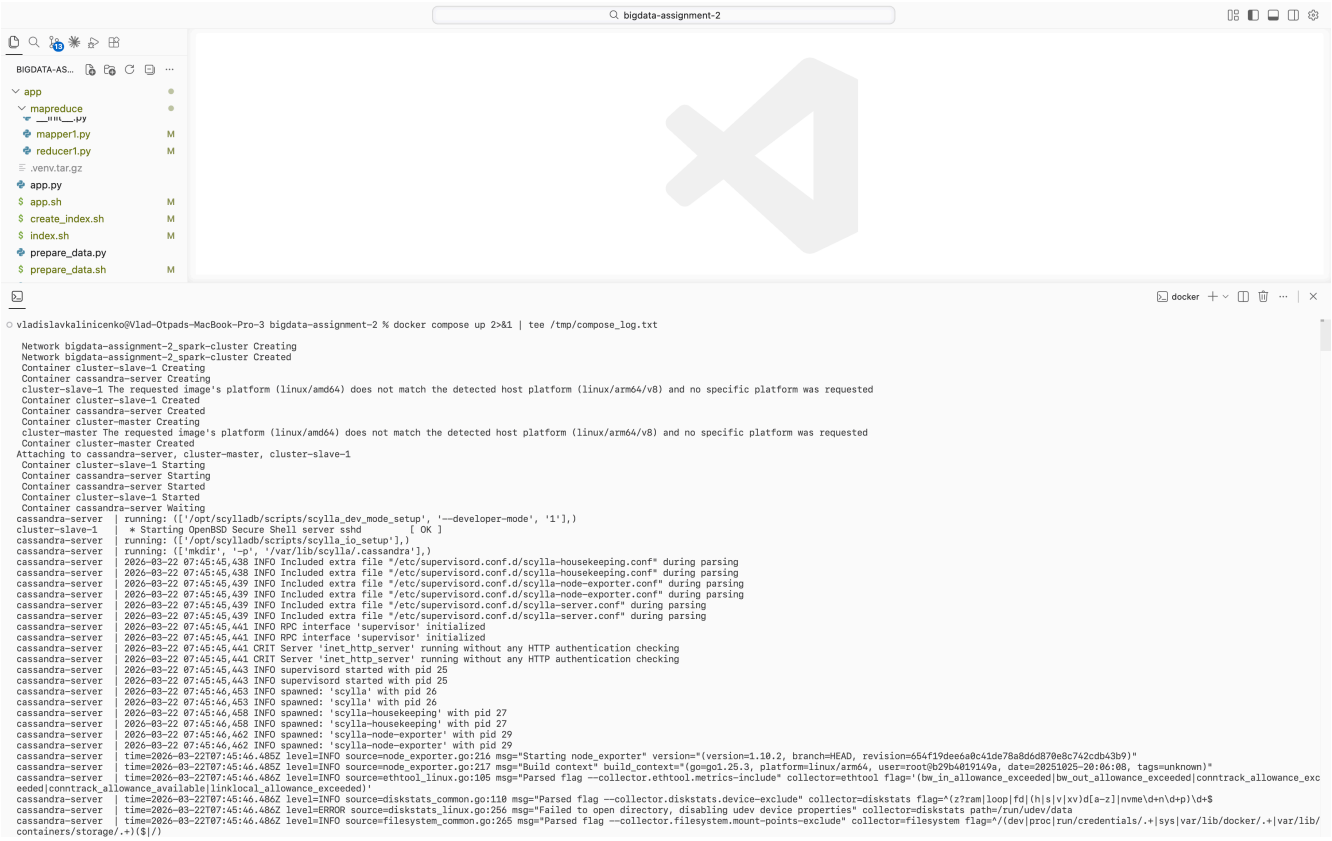

Figure 1: Docker Compose Up

Docker Compose – Services Started All three containers started and the pipeline began running.

HDFS NameNode Web UI – Port 9870 One live DataNode, 1,263 files and directories, 742 MB stored.

YARN ResourceManager Web UI – Port 8088 Two applications: the MapReduce streaming job finished with SUCCEEDED, Spark BM25Search submitted.

MapReduce Job – Indexing Complete Hadoop Streaming job completed. 1,000 map input records, 239,324 reduce output records written to /indexer/part-00000.

ScyllaDB – Index Verification `SELECT COUNT(*) FROM search_engine.inverted_index` returns 239,324 rows – unique term–document pairs.

global_stats: N=1000, dl_avg=558.288.

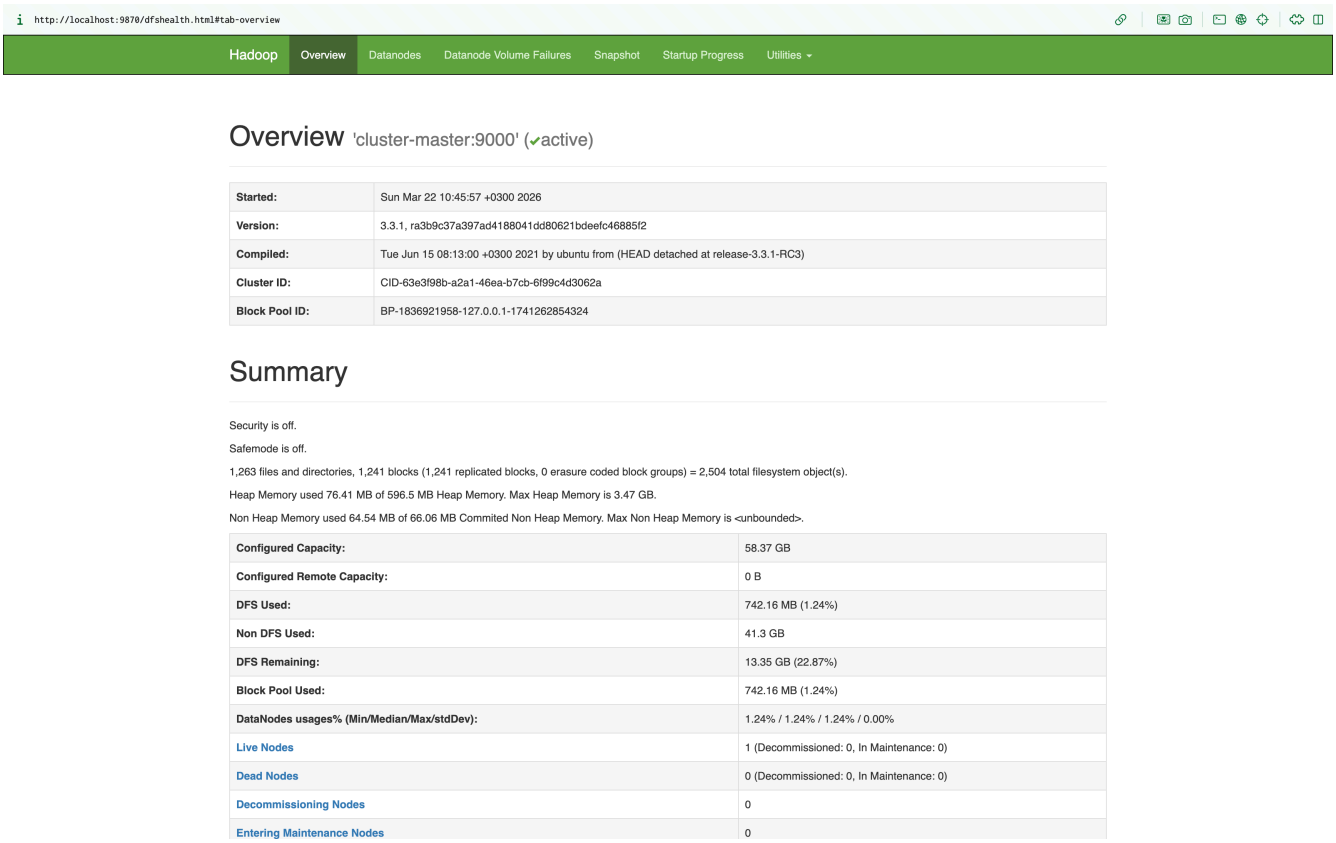

Figure 2: HDFS UI

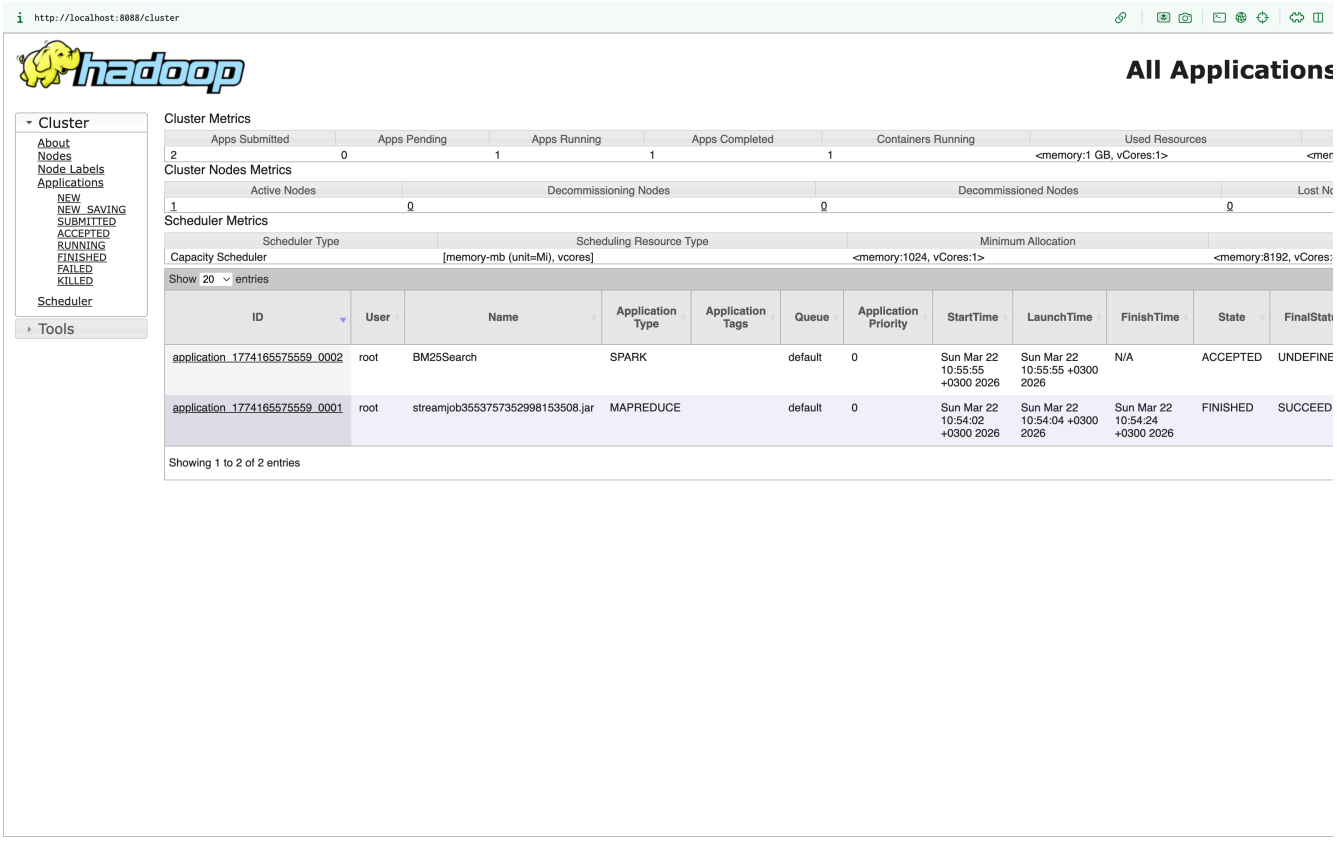

Figure 3: YARN UI

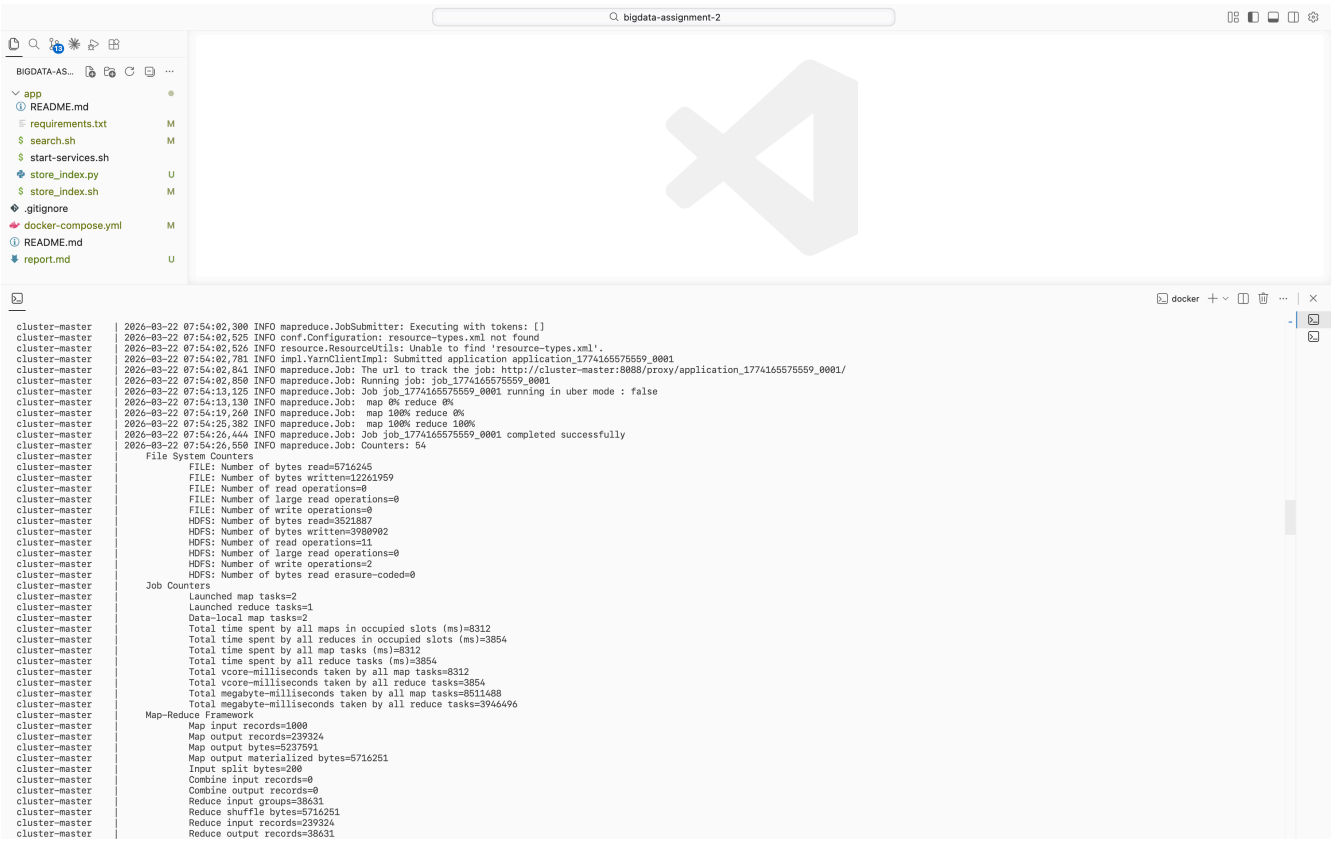

Figure 4: MapReduce Success

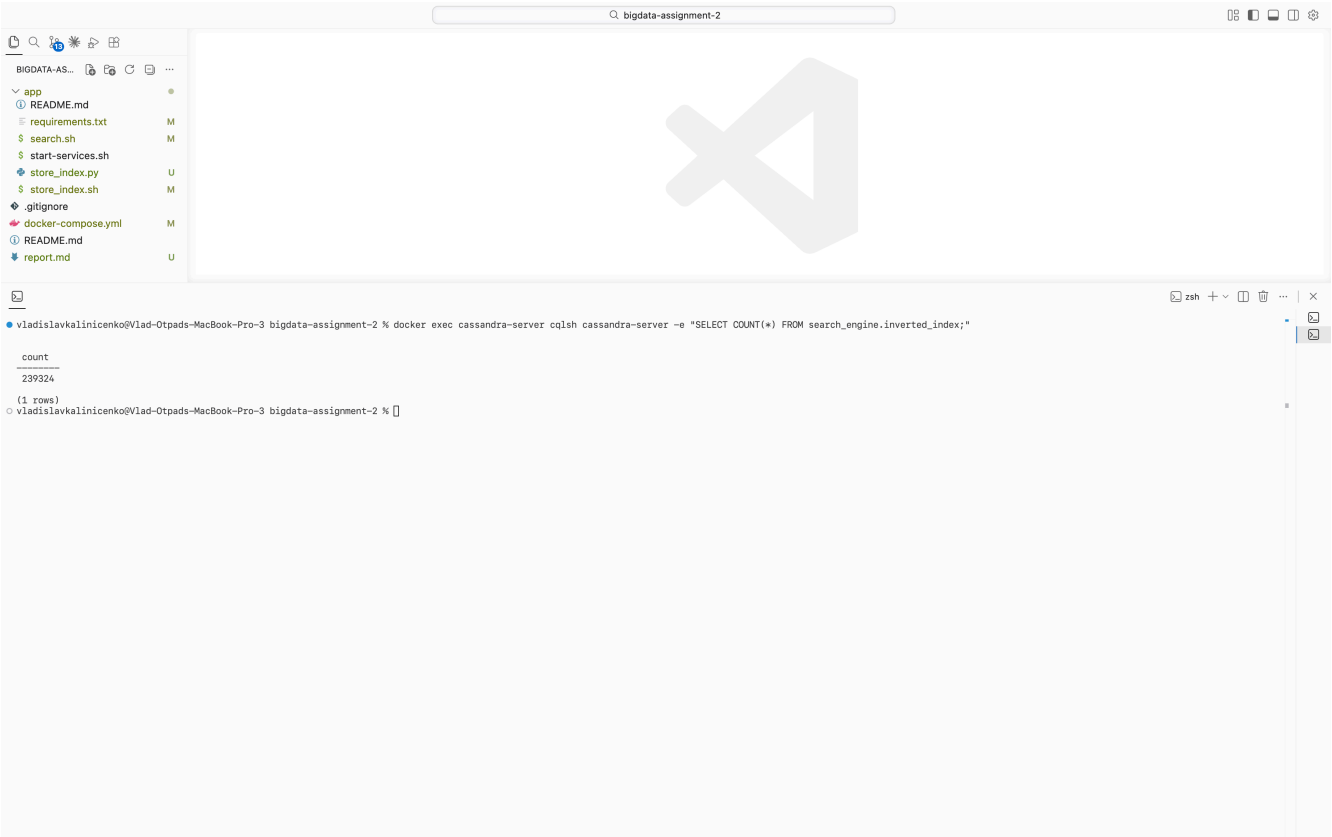

Figure 5: ScyllaDB Count

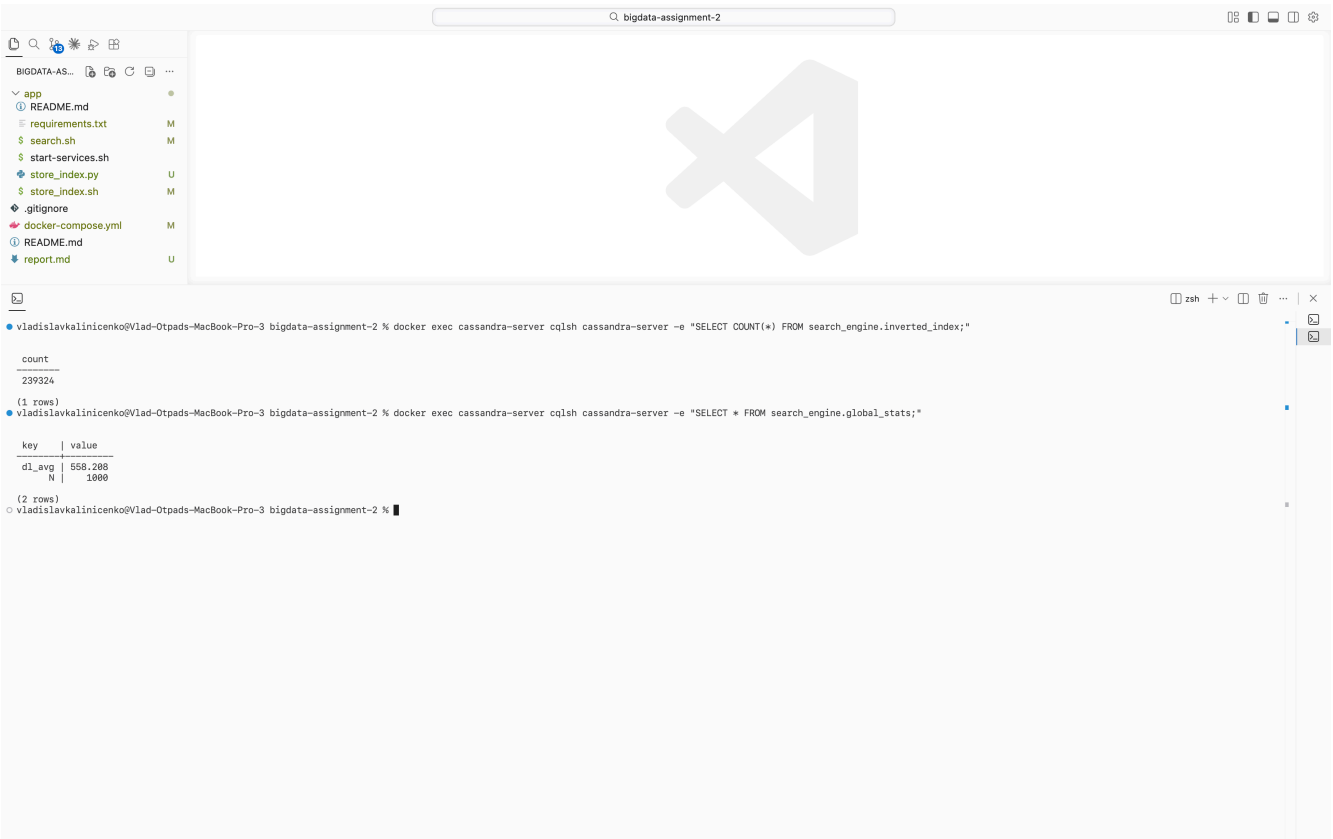

Figure 6: ScyllaDB Global Stats

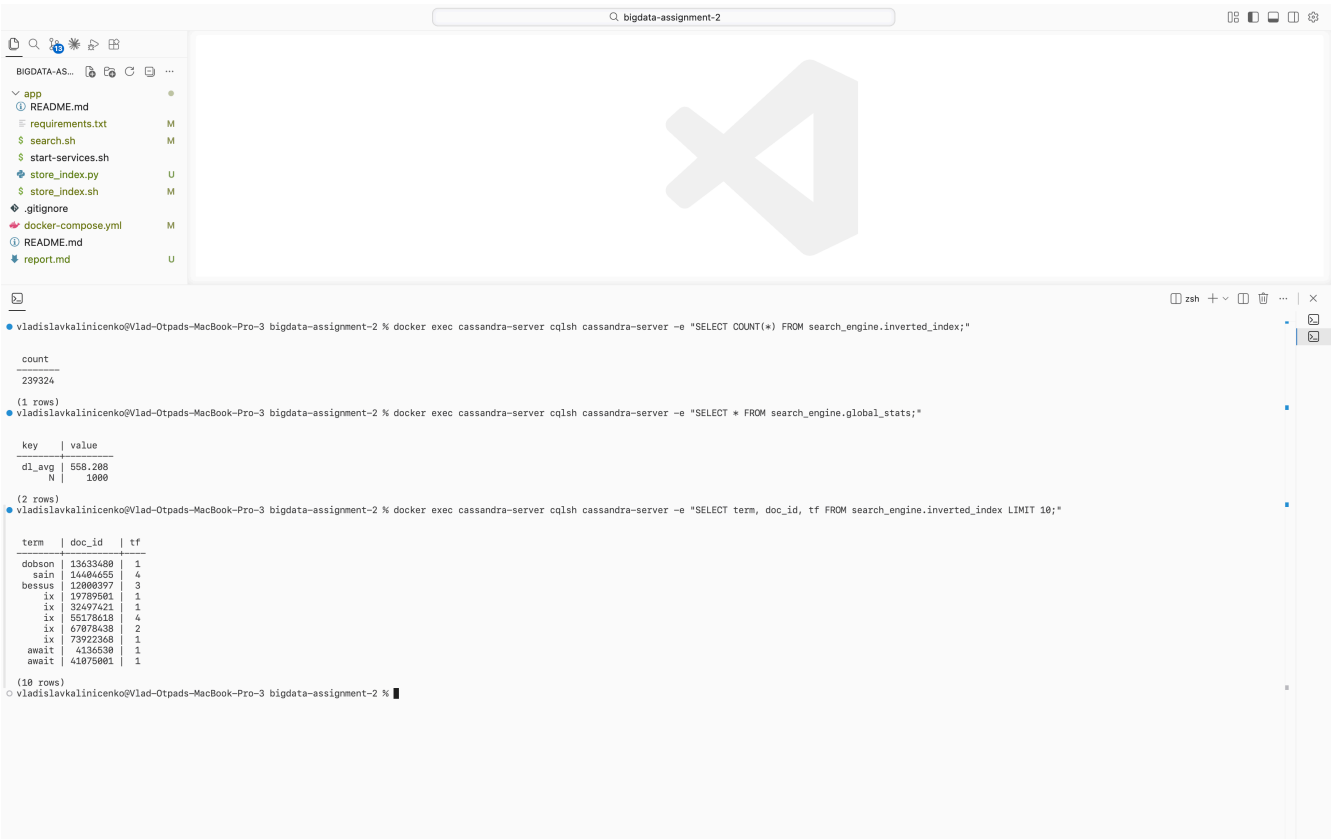

Figure 7: ScyllaDB Sample Rows

Ten sample rows from inverted_index.

Search Query Results Query: “world war history”

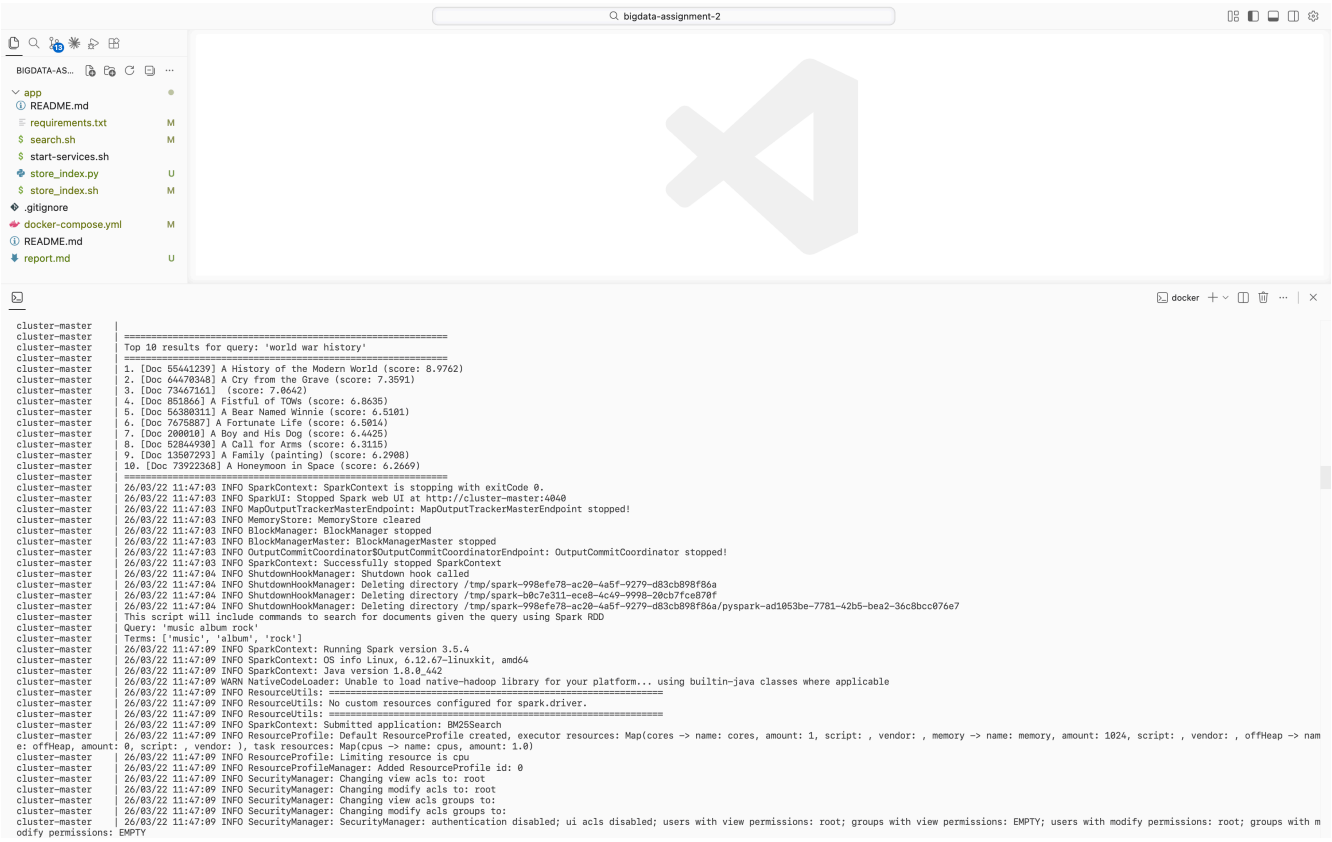

Figure 8: Search world war history

Top results are Wikipedia articles about war and historical conflicts.

Query: “music album rock”

Results include articles about rock bands and music albums.

Query: “united states president”

Top results are articles about US presidents and American political history.

Reflections

BM25 gave pretty good results across all three queries. Articles where multiple query terms appear with high frequency consistently ranked at the top, and the top results made sense in all three cases.

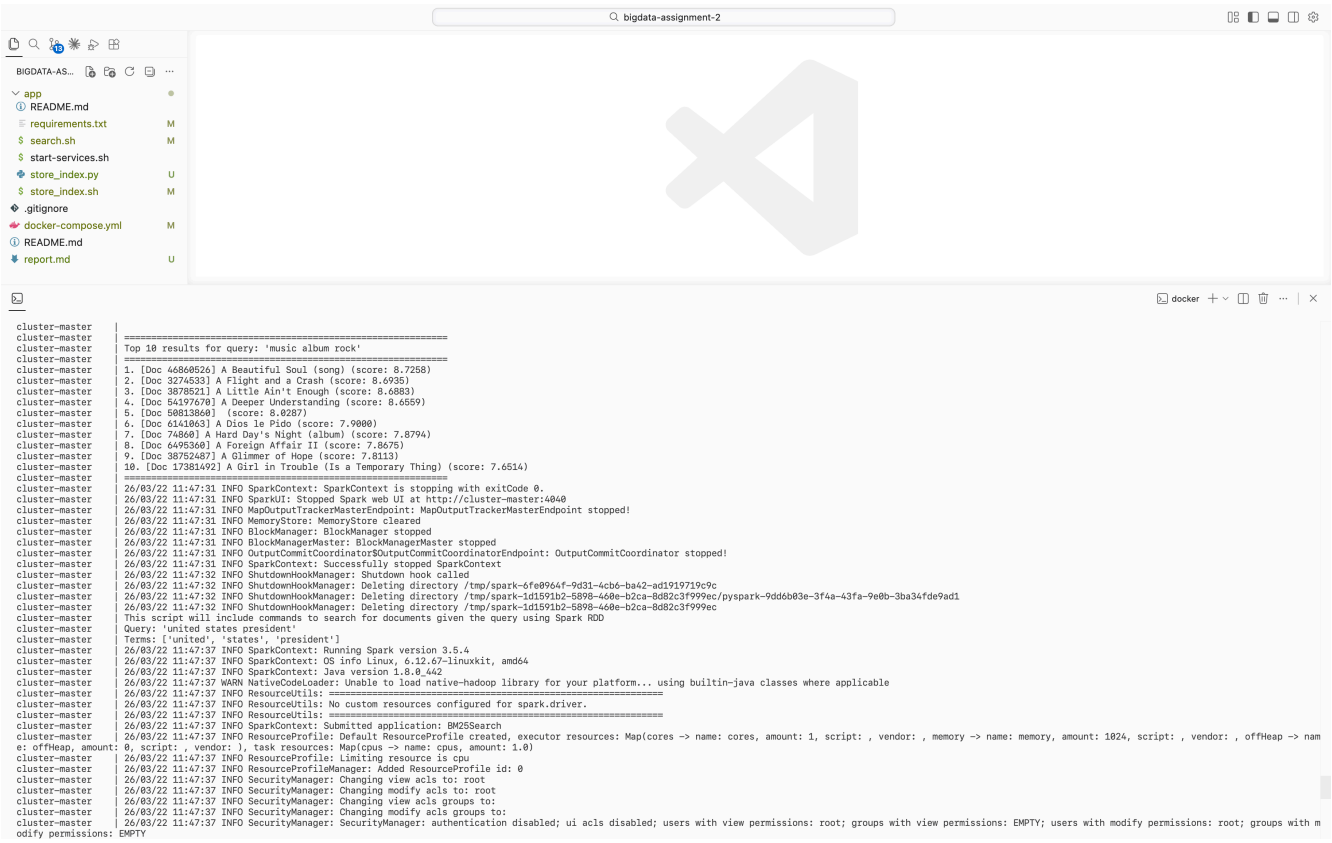

Figure 9: Search music album rock

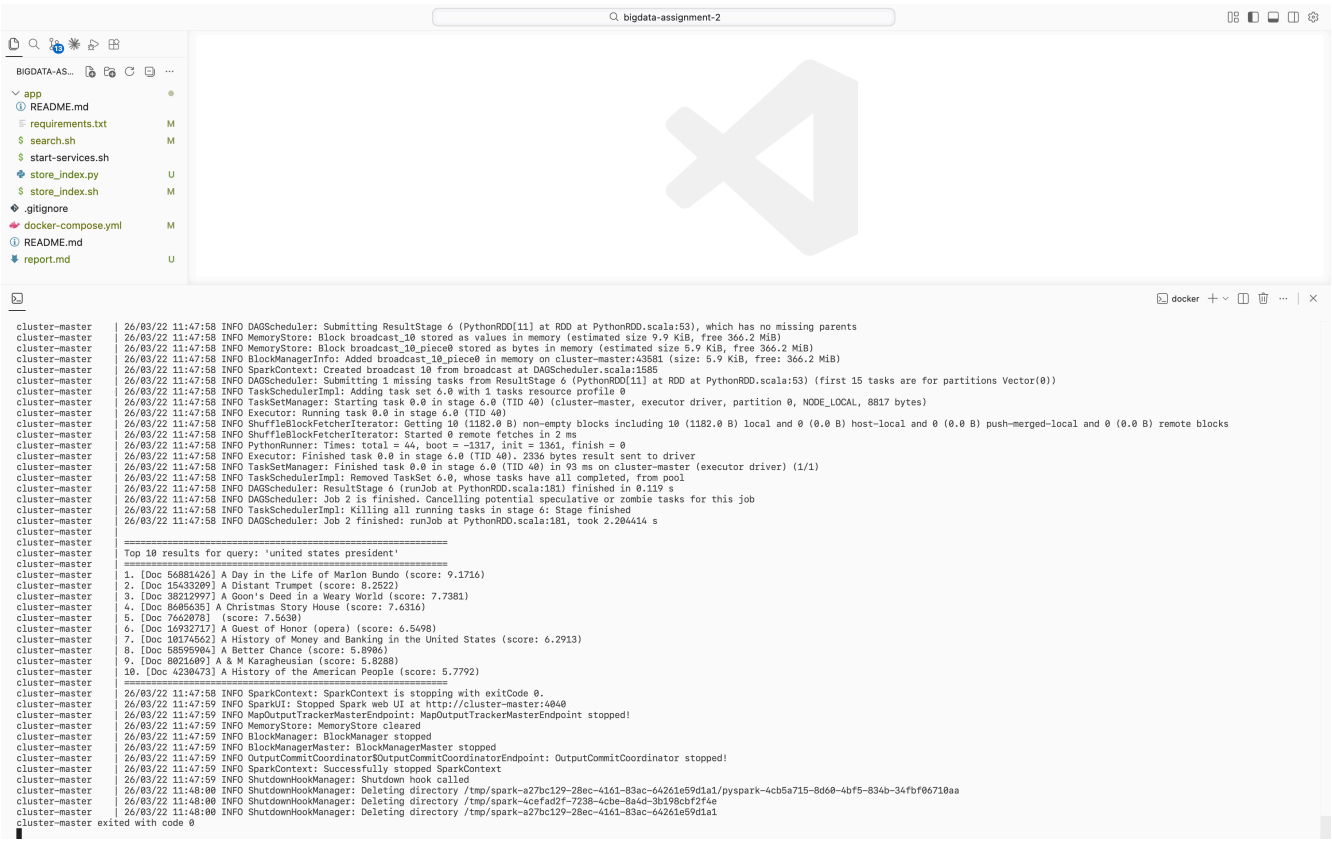

Figure 10: Search united states president